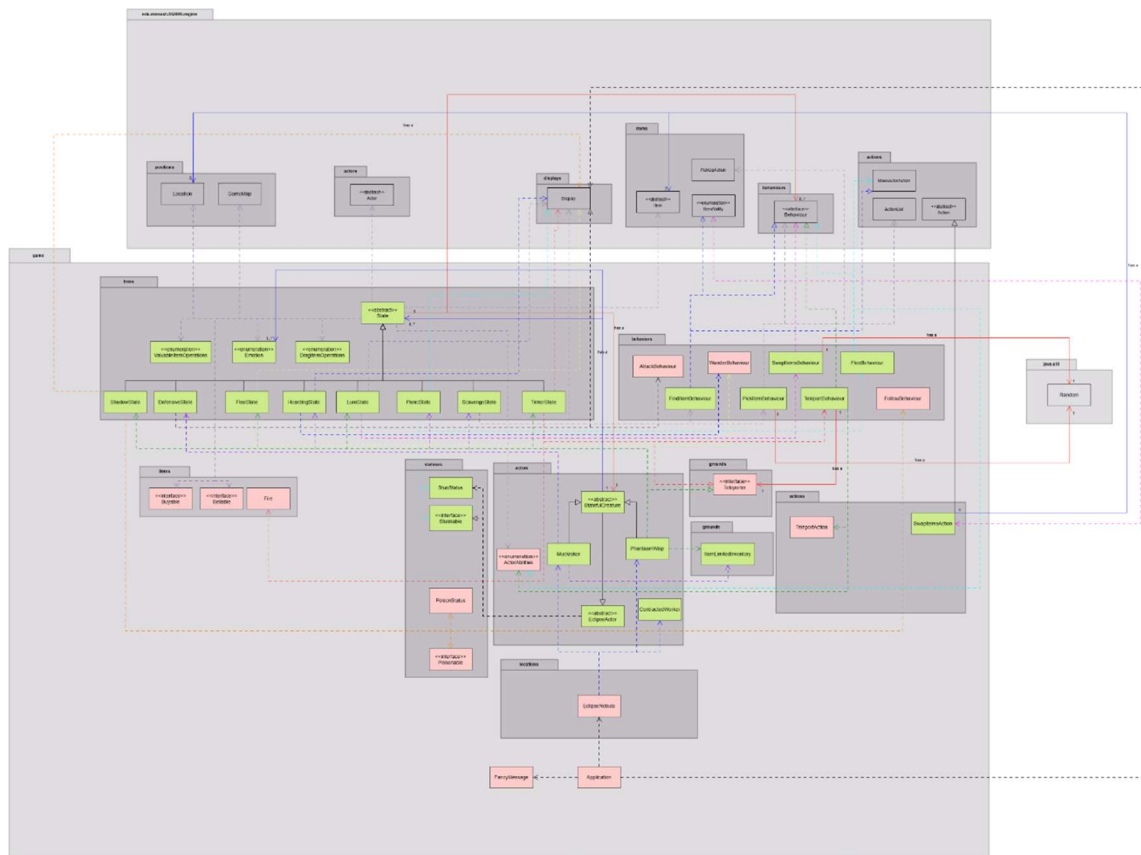
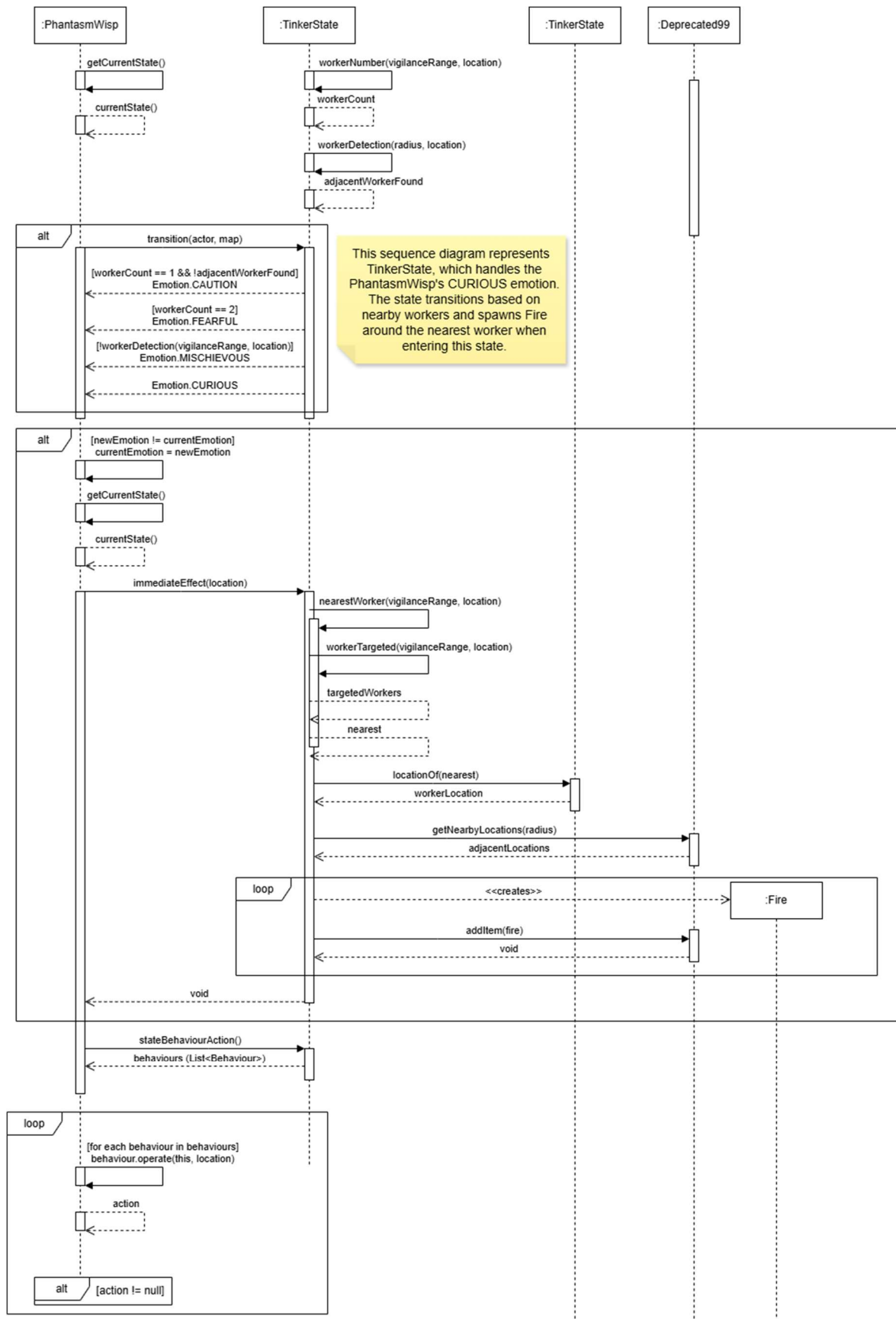


REQ 5 Design Rationale

UML



Sequence Diagram



Design 1: Emotion Based State Pattern

Transition logic lives in `game.states` (`ScavengeState`, `PanicState`, `LureState`, etc.), orchestrated by `StatefulCreature` and concrete actors `Muckraker` and `PhantasmWisp`. Each creature registers four State objects in a `TreeMap<Emotion, State>`. `playTurn` calls `transition`, updates `currentEmotion` when it changes, runs `immediateEffect`, then executes that state's behaviours. Each state class owns deterministic rules (worker count, adjacency, inventory fullness) and entry effects (steal valuable item, stun via `StunStatus`, push/pull ground items).

every creature has four distinct states, non-random transitions driven by surroundings, each state can reach multiple successors (e.g. `ScavengeState` → `Curious`, `Greedy`, or `Fearful`), and transitions fire immediate complex effects separate from turn behaviours. **Hypothetical example:** a `Muckraker` with a full inventory and no workers nearby leaves `Scavenge` and enters `Hoard`; on entry it drops a random inventory item while later turns wander. **Maintainability:** new moods add a class plus one `addNewState` line without rewriting other states. **Scalability:** `PhantasmWisp` reuses the same framework with different transition geometry (adjacent vs 2–4 tile bands). **DRY** centralises worker scanning in `State`; **KISS** keeps `StatefulCreature` a thin coordinator. **SOLID: SRP** splits each mood's rules; **OCP** extends by new `State` subclasses; **Liskov** substitutability via the shared `State` API. **Connascence: of name** links each `Emotion` enum constant to its `State` class (moderate—rename requires coordinated edits); **of type** binds every state to `StatefulCreature` through the constructor (strong—states cannot attach to arbitrary actors). Together these tighten compile-time safety but couple states to this actor family. **Extension:** add `GluttonState` extends `State` for a fifth greedy variant on `Muckraker`; register it in the constructor (association). On transition to `Glutton`, `immediateEffect` could consume a ground item while `GrowBehaviour` handles turns—`StatefulCreature` still drives the pipeline.

Pros	Cons
Isolated state classes make transition maps easy to trace and test per mood.	Eight state classes increase navigation overhead for small changes.
Shared <code>State</code> helpers avoid duplicating worker detection across eight states.	Emotion-to-class mapping is scattered across two actor constructors.
Both creatures share one lifecycle without copy-pasting <code>playTurn</code> logic.	Strong type binding to <code>StatefulCreature</code> limits reusing states on non-stateful actors.

Design 1: Alternative Design

A **monolithic actor** would store `currentState` as an `int` or `enum` and implement all transitions and effects inside `Muckraker/PhantasmWisp` with large switch blocks and private helper methods. **Hypothetical example:** entering panic in one switch branch would stun neighbours inline instead of delegating to `PanicState.immediateEffect`. **Maintainability** degrades as each new mood lengthens the same file. **Scalability** across two creatures duplicates switch trees. **DRY** suffers unless helpers are extracted anyway; **KISS** looks simpler initially but hides growing complexity. **SOLID: SRP** violated when one class owns eight transition graphs; **OCP** closed—every new mood edits the actor. **Connascence comparison:** the alternative raises **of position** (transition order inside one method) and **of execution** (shared mutable fields touched by every branch), generally **stronger** than Design 1's **of name** enum map. Weaker type connascence per class, but overall coupling is higher because any edit risks unrelated states. Trade-off: fewer files vs fragile mega-classes. **Extension:** `MuckrakerStateHandler` as a separate class still associated with `Muckraker`; the actor delegates to it—partial recovery of SRP at the cost of an extra layer.

Pros	Cons
Fewer types to open when debugging a single creature.	Eight moods for two creatures invite duplicated predicates and copy-paste bugs.
All transition rules visible in one scrollable unit.	Mixing transition, effect, and behaviour logic weakens cohesion.
No abstract State hierarchy to learn.	Refactoring one mood risks regressions in unrelated branches.

Design 2: Prioritized Engine Behaviour Composition

Per-turn movement and item interaction are delegated to engine `Behaviour<Actor, Action>` implementations in `game.behaviours`, registered inside each State via `addNewBehaviour(priority, ...)`. `ScavengeState` wires `PickItemBehaviour` and `FindItemBehaviour`; `PanicState` uses `FleeBehaviour`; `LureState` uses `SwapItemsBehaviour`; `TinkerState` uses `TeleportBehaviour`. `StatefulCreature` iterates behaviours in priority order after transitions—matching FIT2099's intended engine usage.

At least two states per creature expose **new** ground-level behaviours (pickup chain, flee, swap, teleport) with non-trivial logic, while transitions and `immediateEffect` remain in state classes. **Hypothetical example:** in `Curious TinkerState`, `TeleportBehaviour` relocates the nearest worker while entry fire on

adjacent tiles runs separately in immediateEffect. **Maintainability:** behaviours are unit-testable without simulating full emotion graphs. **Scalability:** reuse FleeBehaviour on any actor by passing range. **DRY** shares movement algorithms; **KISS** keeps states as declarative wiring lists. **SOLID: SRP** separates “how to act” from “when to change mood”; **ISP**—states depend only on Behaviour’s operate. **Connascence: of meaning**—priority integers must align with intended precedence (moderate); **of type** between Behaviour and Action (weak, engine-standard). **Extension:** add ConfuseBehaviour and register it in a new ChaosState at priority 1; ChaosState depends on ConfuseBehaviour (dependency), collaborates through stateBehaviourAction() like existing states.

Pros	Cons
New behaviours plug in without rewriting transition code.	Worker-search logic is duplicated between FleeBehaviour and State helpers.
Aligns with engine patterns markers expect for NPC turns.	Priority numbers are implicit contracts not enforced by the type system.
Behaviours can be recombined across creatures and future actors.	Debugging requires tracing both state transition and behaviour chain each turn.

Design 2: Alternative Design

Inline behaviour methods inside each State subclass (e.g. private Action tryPickup(Actor, Location)) would replace separate behaviour classes. **Hypothetical example:** ScavengeState would call an internal pickup helper instead of PickItemBehaviour. **Maintainability** collapses when two states need similar movement—copy-paste or fat base class. **Scalability** to a third creature reimplements flee logic again. **DRY** worse than Design 2 unless lifted late; **KISS** fewer packages. **SOLID: SRP** blurs state as mood controller and action generator. **Connascence comparison:** inline design shows **stronger of execution** (shared private fields in one state object) versus Design 2’s **weaker of type** across small behaviour classes—Design 2 minimises coupling but adds files and indirection. Trade-off: lower class count vs harder reuse. **Extension:** extract ItemSwapLogic utility called from multiple states—hybrid that slowly recreates behaviour objects.

Pros	Cons
Single file per mood shows both transition and action code together.	Duplicated flee/pickup code across states increases defect risk.
No indirection through behaviour priority maps.	State classes grow until they violate SRP.
Slightly fewer classes in the diagram.	

Comparison of Alternative Designs

The monolithic-switch alternative concentrates **connascence of position and execution** in one actor type, so a small requirement tweak can ripple through unrelated moods. The inline-behaviour alternative reduces class count but reproduces **connascence of algorithm** across state subclasses whenever two moods share movement. Between alternatives, the monolithic handler delegate weakens execution connascence slightly yet retains centralised branching; inline methods fare worst for reuse. The behaviour-delegation alternative (not chosen) would mirror Design 2 but at the cost of more abstraction layers—best coupling profile, highest file overhead.

CONCLUSION

The diagram above shows the final design of REQ 5, which utilizes Design 1 (emotion-keyed State objects coordinated by StatefulCreature) together with Design 2 (prioritized engine Behaviour classes for turn actions). Design 1 was chosen because it cleanly separates deterministic transition graphs and immediate effects for Muckraker and PhantasmWisp without procedural sprawl. Design 2 was chosen to host novel behaviours (PickItemBehaviour, FleeBehaviour, SwapItemsBehaviour, TeleportBehaviour) while keeping transition predicates in state classes—balancing **KISS** in the turn loop with **DRY** reuse of engine interfaces. **SOLID** alignment (**SRP**, **OCP**, **Liskov** on State) supports the eight moods across two creatures within current scope. **Connascence of name** between Emotion and State is an acceptable trade-off: it is explicit, searchable, and weaker than mega-switch **execution** coupling, which aids long-term maintenance. Trade-offs include duplicated worker scanning between FleeBehaviour and State helpers and eight state source files to navigate. Monolithic or fully inline alternatives may suit larger teams or engines without behaviour lists, but would sacrifice modularity gained here.